Chrome Tab Recorder: Technical Overview

Tech stack decisions

I started by researching third party APIs and choosing the stack. The extension uses WXT, React and TypeScript. The backend is Python with FastAPI on Google Cloud.

Transcription: AssemblyAI

AssemblyAI was selected because it is the only provider whose limits and job model fit a two hour recording without workarounds. Its /v2/transcript endpoint accepts files up to 5 GB and 10 hours in a single request, so no client side segmentation or transcript stitching is needed. Completion is delivered by webhook with up to 10 retries, and we also schedule our own status checks as a backup. That maps directly onto the queue based backend.

AssemblyAI can also tell speakers apart. Our recorder helps it do this by saving the tab's audio on the left channel and your microphone on the right channel. Because the two sources are kept physically separate, the transcript can always tell which lines came from the tab and which came from you. On top of that, AssemblyAI looks inside the tab channel and labels the different people speaking there (for example, several participants on a call). So you get both "tab vs. you" and "who on the tab was talking".

The following providers were not selected:

- **Deepgram:** any request that takes over 10 minutes to process returns a 504 timeout.
- **OpenAI (Whisper):** 25 MB upload cap, so it cannot support 2 hours without chunking the audio, and it has no async job model.
- **Google Chirp 3:** batch mode is documented for audio up to about 1 hour (20 minutes with word timestamps). Its speaker separation only runs on single channel audio, so you cannot get both the tab/mic split and speaker labels inside the tab channel from one request.

The big picture

The system has two halves that share only one thing: a job ID the extension creates the moment you press record, plus the Google Drive file ID once the upload finishes.

The extension owns everything until the recording is safely in the user's Drive. The backend owns everything after that. I drew the line there on purpose. A two hour recording is a few gigabytes, and I never wanted that passing through my servers. Recording should never depend on the network. And transcription takes time, so anything that has to outlive the browser runs on the backend.

Four rules hold across the whole system:

1. **The video never passes through our servers.** The browser uploads it straight to Drive. The backend only ever handles a small extracted audio track (40 to 70 MB for two hours), which is deleted after 7 days.
2. **Nothing ever waits on a slow operation.** Every backend step finishes in seconds or minutes, and transcription results arrive by webhook.
3. **Job state only moves forward.** One state machine is written in both TypeScript and Python, and both pass the same 36 test cases. The extension and backend can never disagree, and a late or repeated message can never undo progress.
4. **Everything is keyed on the job ID.** That is what makes every step safe to retry.

1. Chrome extension architecture

Manifest V3 splits an extension into separate pieces, so each one has a single job.

- **Service worker:** the coordinator. It routes messages, sets the REC badge, starts the recorder, runs the upload queue and handles crash recovery. Chrome kills it whenever it likes, so it keeps nothing in memory. Everything lives in IndexedDB or chrome.storage.
- **Offscreen document:** the actual recorder. Service workers cannot use media APIs, so Chrome provides this hidden page as the official way to run getUserMedia and MediaRecorder.
- **Popup:** the Record Tab button, audio only toggle, tab audio toggle, microphone toggle, status indicator, sign in and recent jobs. Its click is the user gesture Chrome requires before it will capture a tab.
- **Dashboard:** the full job queue and history, with live status and Retry buttons.
- **On page widget:** a Loom style ball. Click it to start; it turns into a pill with a timer and stop button while the page gets a red border. It lives in a closed shadow root, so the website's CSS cannot break it.
- **Permissions page:** a one time microphone prompt, because that prompt cannot appear inside the hidden offscreen page.

Tab audio and microphone are both on by default, and either can be switched off before recording starts.

Every job in the queue shows:

- its name
- its creation time
- its current status
- its retry state (for example "attempt 3, next try in 2m")
- Drive and Doc links once they exist

The statuses are the ones in the brief: Pending upload, Uploading, Uploaded, Queued for transcription, Transcribing, Completed and Failed. There is also a "Sign in needed" pause. History lives in IndexedDB, so it survives extension reloads and browser restarts.

The extension ID is pinned in the manifest, so the Google sign in redirect is the same on every machine.

2. Tab recording implementation

When you press record, the popup asks Chrome for a stream ID for the current tab. The tab is looked up when the popup opens, so nothing delays the call and Chrome still counts it as the user's click. The service worker makes sure the offscreen document exists and passes the stream ID along.

The offscreen document then:

1. Opens the tab's video and audio, plus the microphone with echo cancellation if it is on.
2. Mixes the audio with the Web Audio API into one stereo track: tab on the left channel, mic on the right. If only one source is on, it records mono.
3. Sends the tab audio to your speakers, so you can still hear what you are recording.

MediaRecorder encodes WebM: VP9 video (VP8 as a fallback) at about 3 Mbps, capped at 1080p30, and Opus audio at 128 kbps. It hands over a chunk every 5 seconds. Audio only mode simply leaves out the video track.

If the tab is closed mid recording, the recorder stops cleanly with what it has. The "currently recording" state lives in chrome.storage.session and the toolbar badge, never in the popup, because the popup disappears as soon as it loses focus.

3. Local persistence strategy

The extension uses two main stores: OPFS for the video and IndexedDB for job records.

OPFS is a private file system the browser gives each extension. It is on real disk, but the user cannot see it in Finder. Every 5 second chunk is written as its own file (`/recordings/{jobId}/seg-000000.webm`, `seg-000001.webm` and so on) and closed straight away.

This matters because OPFS only saves a file when it is closed. One long file would save nothing until the end. One file per chunk means:

- a crash loses 5 seconds at most
- memory never holds more than one chunk, however long you record

On stop, a small manifest records the size, duration and channel count. The chunks are one continuous stream split at byte boundaries, so joining them is just concatenation. That happens lazily, so the full file never sits in memory.

IndexedDB holds one small record per job: status, sizes, Drive and Doc IDs, retry bookkeeping and a done flag for each step. It is both the durable job queue and the history the user sees, and it survives reloads and restarts. I chose it over chrome.storage.local, which is a 10 MB key value store with no indexes or transactions.

Crash recovery runs on startup and every minute. Any job still marked “recording” with no live session is salvaged:

- chunks on disk: it moves to Pending upload
- no chunks: it is marked Failed

I tested this by force quitting Chrome mid recording.

What is stored locally and for how long:

- The video is deleted about 10 minutes after the upload is verified, meaning Drive reports the same size as the local bytes. The deletion doubles as an integrity check.
- Job history is kept forever.
- The backend only ever reads from Drive, so deleting the local copy is safe.

Before recording, the extension checks free disk space. It warns if there is not room for a two hour recording and refuses below 500 MB. It also holds the unlimitedStorage permission, so Chrome will not evict recordings when the disk is under pressure.

4. Upload retry logic

After recording stops, the job is created locally and the upload starts automatically. Recordings go into a “Tab Recorder” folder the extension creates in the user’s Drive on the first upload. When the upload finishes, the Drive file ID is saved on the job record, and that ID is what the backend receives.

Uploads run from a durable queue in the service worker.

When it runs: jobs go oldest first. The queue wakes on:

- a one minute alarm
- browser startup
- the network coming back
- a recording finishing
- a manual retry

One upload per job: a short lock in chrome.storage.session (90 seconds, renewed after every chunk) makes sure two wake ups never upload the same job. If the holder crashes, the lock simply expires.

Backoff: failures wait from 30 seconds up to 30 minutes, with a small random jitter. The schedule is saved on the job, so it survives restarts.

Limit: after 10 automatic attempts the job shows “Stopped after 10 tries”. The Retry button is always available.

Expired sign in: a 401 from Google triggers one token refresh and a retry of the same request. If that also fails, the job pauses as “Sign in needed”, and signing in again resumes every paused job.

Progress shows live in the queue, and success is confirmed when the job moves to Uploaded with a Drive link.

5. Duplicate job prevention

Every layer is keyed on the job ID, and each has an automated test.

1. **Session reuse:** an interrupted upload resumes its saved session instead of starting a new one.
2. **Adopt, don't upload again:** if a session has died (Google expires them after a week), the uploader first searches Drive for a file tagged with this job ID and adopts it. *Tested: expired session, existing file adopted, nothing uploaded twice.*
3. **Registration:** the job ID is the Firestore document ID, so registering twice returns the existing record. *Tested: register twice, one record, one task.*
4. **Pipeline tasks:** Cloud Tasks names are built from the job ID, so a duplicate enqueue is rejected.
5. **Transcription:** the transcript ID is written in a transaction that first checks it was empty, so a job is never submitted twice.
6. **Google Doc:** the Doc is found by job ID before one is created. A half written Doc is cleared and rewritten, never appended to.
7. **Email:** gated by a "sent" marker. *Tested: finalize twice, one Doc, one email.*

Two tiny windows remain, and I left them open on purpose:

- A crash in the moment between Gmail sending and the marker being saved could produce one duplicate email.
- A crash between AssemblyAI accepting a job and the ID being saved could leave one orphaned transcript (about $1).

Closing them would cost more complexity than they are worth.

6. Queue system choice

I chose Cloud Tasks, with one queue per pipeline step: extract, transcribe submit and finalize. Separate queues mean a burst of retries in one step never blocks another.

I considered two alternatives:

- **Pub/Sub:** push subscriptions must be acknowledged within 10 minutes, and they cannot deduplicate by name or schedule a message for later. Cloud Tasks gives all three: a 30 minute delivery deadline that matches the worker's timeout, named tasks for dedup, and scheduled tasks, which is exactly how the transcription backup check is built.
- **Cloud Workflows:** this would add a second orchestration language for a pipeline that is a straight line. The job record in Firestore already holds the orchestration state.

How a task works:

- **Enqueue:** each task is named from the step and the job ID. If that name already exists, Cloud Tasks rejects it and we treat that as success, so racing events collapse into one piece of work.
- **Delivery:** Cloud Tasks calls the private worker with a signed identity token, which Cloud Run checks before my code runs.
- **Retries:** success means the task is done; anything else retries, up to 8 times with backoff from 10 seconds to 10 minutes. Temporary problems such as a Drive hiccup return a 503 so the queue retries. Permanent ones such as revoked access mark the job Failed and return success, so the queue stops and the user gets a Retry button instead of pointless redelivery.
- **If all 8 attempts fail:** the Retry button bumps a generation number on the job, which gives the tasks fresh names, and restarts from the first missing step. Everything already produced is reused.

7. Transcription orchestration

The pipeline is three separate tasks, each safe to rerun.

1. **Extract:** the worker streams the video from Drive in 16 MB pieces (retrying each piece on server errors) straight into ffmpeg. ffmpeg throws away the video and writes a compressed stereo audio file. The video never touches the server's disk. In a two hour benchmark, extraction took 43 seconds with ffmpeg using about 16 MB of memory. The 40 to 70 MB audio file goes to Cloud Storage.
2. **Submit:** creates a 24 hour private link to the audio and sends it to AssemblyAI. Multichannel and speaker labels are on, and a webhook URL carries the job ID and a secret header. The transcript ID is saved and the first backup check is scheduled. Then the task exits.
3. **Completion** arrives in one of two ways:
   - **Webhook (primary):** it checks the secret, queues the finalize task and replies within AssemblyAI's 10 second window.
   - **Scheduled checks (backup):** these start at 15 minutes and space out to an hour, up to 12 times (about 10.5 hours in total). If they run out, the job fails with a timeout and a failure email.

Both paths queue the same named task, so whichever arrives first wins. A late webhook for an old transcript is ignored.

The extension polls the backend for status and applies it through the same forward only state machine, so repeated updates are harmless.

8. Google Docs generation

The finalize task first finds or creates the Doc in the same “Tab Recorder” folder as the recording, tagged with the job ID so a retry finds it instead of creating another. The Doc ID is saved right away.

The Doc contains:

- a title
- the recording date, time and duration
- a link to the recording in Drive
- the full transcript, one paragraph per speaker turn, each with a bold speaker label and a timestamp

Stereo recordings show “Tab audio” and “You (mic)”. Speakers inside the tab channel are labelled too, for example “Tab audio, Speaker A”.

One detail worth noting: the Docs API counts positions in UTF 16 code units, so an emoji in a tab title would shift every later insert. All positions are calculated in those units, and there is a regression test with an emoji title. Requests go in batches of up to 100.

A two hour transcript is around 100 to 150 thousand characters, comfortably one Doc. It is created with the user’s own Google permission, so they own it and no sharing step is needed.

9. Email notification pipeline

The same finalize task then sends the email through the Gmail API, as the user and to the user. It needs no email provider or domain setup, and it arrives like any message the user has sent themselves. It contains the recording name, the Drive link, the Doc link and the duration.

- **At most once:** a “sending” flag is saved before the call and a “sent” marker after. Finalize skips the step if the marker exists.
- **Failure emails:** if a job fails permanently, one failure email is sent.
- **Fallback:** the sender sits behind a small interface, with Resend available as a drop in alternative.

10. How the system handles large files

A two hour recording at 1080p is nominally 2.5 to 3.7 GB. Real content compresses better: my measured rate was about 13 MB per minute.

The usual approaches break on files this size:

- holding it in memory crashes the browser
- sending it through a server hits request limits (Cloud Run caps requests at 32 MB)
- synchronous transcription APIs time out

- anything that waits for a long job holds a connection open longer than any platform allows

This design never holds more than a few MB of the recording at once, and never waits:

- **Recording:** one 5 second chunk in memory at a time.
- **Upload:** browser straight to Drive in resumable pieces.
- **Extraction:** streamed through ffmpeg and never stored. A 2 GB worker has two orders of magnitude of headroom.
- **Transcription:** one async request.
- **Output:** one Doc and one email.

Validation. I recorded a real two hour session: a podcast playing in a tab, with me talking over it from time to time, left to run unattended. It uploaded and transcribed with both podcast hosts separated as Speaker A and B on the tab side, my comments tagged as "You", and the email arrived. Backend time from registration to email was 5 minutes 20 seconds.

I also:

- ran a 1.2 GB synthetic file through the pipeline with the network cut mid upload (it resumed from the same byte and left one Drive file)
- recorded a real 60 minute session to measure disk usage per minute
- benchmarked two hour extraction

All the numbers are in `docs/VALIDATION.md`.

11. Resumable or chunked upload strategy

Uploads use Google Drive's resumable protocol:

1. The extension asks Drive for an upload session, tagged with the job ID, and saves the session URL to IndexedDB before sending any data.
2. It sends the file in 8 MB pieces (Drive requires multiples of 256 KB). After each piece, Google replies with how many bytes it has confirmed, and that number is saved before the next piece goes.
3. If anything interrupts the upload, on resume it first asks Google how much it actually has, rather than trusting its own notes, because the piece in flight may have partly landed. It then continues from exactly that byte.

This covers a network drop, a killed service worker, a closed browser and a rebooted machine. On a 1.2 GB test I cut the Wi-Fi at 6% and it resumed from 6%.

If the session itself has expired, the dedup search in section 5 adopts any file that already finished.

12. Handling long audio transcription

Long audio needs no special handling:

- A two hour file is a single request to AssemblyAI, so nothing is split. That matters because splitting breaks speaker labels at the cut points.
- Submission returns in milliseconds. While AssemblyAI works, nothing on our side runs or waits; there is just a Firestore record saying “Transcribing”, which the user sees in the queue.
- Completion arrives by webhook, with scheduled checks as a backup for up to about 10.5 hours.

A 10 hour recording would follow exactly the same path.

13. Idempotency strategy

Every backend handler follows one rule: read the job first, and if your step is already done, return success and do nothing. Each step has its own done marker:

- audio stored
- transcript ID saved
- Doc content written
- email sent

Three more things make this safe:

- Every status change in Firestore is a transaction that checks the current state first, so when two handlers race, exactly one wins.
- Named tasks collapse duplicate enqueues.
- The forward only state machine means replays can never move a job backwards.

Every handler is tested by running it twice. On the extension side, every status change goes through the same pure transition function, so applying an update twice produces the same record.

14. Retry logic

Layer	How it retries	Limit	After that
Extension upload	backoff 30 s to 30 min with jitter	10 attempts	Failed, Retry button
Pipeline step	Cloud Tasks backoff 10 s to 10 min	8 attempts	Retry restarts from the first missing step
Transcription check	scheduled checks 15 min to 1 h	12 checks (about 10.5 h)	Failed with timeout, failure email
Manual	Retry button in	any job not	n/a

Layer	How it retries	Limit	After that
	popup or dashboard	completed	

Backend retries always read from the Drive copy, never from the user's machine.

15. Failure handling

Every failure has an error code, a plain English message for the user and a recovery path. The full matrix of 14 scenarios, each with how to reproduce it, is in `docs/ERRORS.md`. The cases the brief names:

Brief case	What happens
Recording succeeds but upload fails	The recording waits on disk and retries automatically.
Upload succeeds but transcription fails	The job is marked Failed with the reason, a Retry button and one failure email.
Expired Google authentication	Jobs pause as "Sign in needed" and resume after signing in again.
Network interruptions and large upload interruptions	The upload resumes from the last confirmed byte.
Duplicate job execution and worker retries	The second run sees the step is done and exits.
Long processing jobs timing out	Cloud Tasks retries, and the step resumes from its saved progress. Transcription itself never holds a connection.
Backend down	The upload still finishes, the job stays at Uploaded, and the extension keeps retrying registration until the backend is back.
Google API errors or rate limits	Retried with backoff, on both the extension and backend side.

A `FAIL_STAGE` setting makes the worker fail on purpose, so retries can be demonstrated and tested.

16. Scaling considerations

- **Storage and bandwidth:** recordings live in each user's own Drive, so our costs do not grow with recording volume. Only the small audio files (deleted after 7 days) and Firestore records do.
- **Compute:** the worker runs one job per instance, since ffmpeg is CPU bound, and Cloud Run adds instances as needed. The API is stateless and cheap, and Cloud Tasks absorbs bursts.

- **Database:** one Firestore document per job, so there are no hot spots and no cross job transactions.
- **Transcription:** AssemblyAI queues anything beyond the account's concurrency limit, which costs time rather than errors.

Known limits, chosen deliberately:

- There is no dead letter queue. A stuck job shows up in the UI with a Retry button instead.
- A retry more than a week after extraction reruns extraction, because the audio has been purged.
- Gmail sending limits do not matter at one email per recording.

How I used GCP

Everything lives in one GCP project and is fully scripted in `infra/setup.sh`, so the whole environment can be rebuilt from scratch with one command.

- **OAuth client:** one Google OAuth client in Testing mode, as requested. It has only three scopes: `openid email`, `drive.file` (the app can only see files it created) and `gmail.send`. The backend exchanges the sign in code using the client secret, which never ships in the extension, and stores the refresh token encrypted in Firestore. The extension only ever holds short lived tokens.
- **Cloud Run:** two services, both built by Cloud Build and stored in Artifact Registry.
  - A small public API that the extension and AssemblyAI talk to, with endpoints for sign in (`/auth/*`), registering and checking jobs (`/v1/jobs`), and the AssemblyAI webhook.
  - A private worker for the ffmpeg and pipeline work. Nothing public can call it; only a dedicated service account is allowed in, enforced by Cloud Run itself.
- **Cloud Tasks:** the job queue, with three queues as described above.
- **Firestore:** job records and encrypted user tokens, with every status change as a transaction.
- **Cloud Storage:** extracted audio files, deleted automatically after 7 days. AssemblyAI gets a private link that expires after 24 hours.
- **Secret Manager:** all five secrets are injected into Cloud Run at startup: the AssemblyAI key, the OAuth secret, the session key, the webhook secret and the token encryption key. Nothing sensitive is in the code.
- **Service accounts:** three of them (API, worker, task invoker), each with only the permissions it needs.

Testing

There are 80 backend and 87 extension automated tests:

- The shared state machine table runs in both languages.

- Every pipeline handler is tested by running it twice.
- The three main dedup guarantees each have a dedicated test.

Manual QA covered:

- crash recovery
- network loss mid upload
- auth revocation
- every toggle combination
- the full two hour run